



UNIVERSIDAD
Blas Pascal

INFORME TÉCNICO TÉCNICAS DE COMPILACIÓN

Asignatura: Técnicas de Compilación

Fecha: 08/6/2026

Autor:

Alessandro Chiavarino

Federico Casani

Profesor: Francisco Ameri Lozano Lopex

1. Introducción

El presente trabajo tiene como objetivo el diseño e implementación de un compilador educativo para un subconjunto del lenguaje C++, utilizando Java y la herramienta ANTLR4. El compilador desarrollado es capaz de analizar programas escritos en dicho subconjunto, verificando su corrección léxica, sintáctica y semántica, además de generar código intermedio y aplicar optimizaciones básicas sobre el mismo.

El proyecto fue desarrollado siguiendo las etapas clásicas del proceso de compilación vistas en la materia: análisis léxico, análisis sintáctico, análisis semántico, generación de código intermedio y optimización. Asimismo, se incorporó visualización del árbol sintáctico y generación de archivos de salida con el código intermedio y el código optimizado.

El objetivo principal del trabajo fue construir una herramienta que permitiera aplicar de forma práctica los conceptos teóricos de compiladores, logrando un sistema modular y extensible, capaz de servir como base para futuras ampliaciones del lenguaje.

2. Análisis del Problema

El problema planteado consistió en desarrollar un compilador para un subconjunto reducido del lenguaje C++, de manera que fuese posible procesar programas simples pero representativos, cubriendo las etapas fundamentales de compilación.

El subconjunto implementado incluye:

2.1 Tipos de datos soportados

- `int`
- `float`
- `double`
- `char`
- `string`
- `bool`
- `void`

2.2 Estructuras y elementos del lenguaje

- Declaración de variables globales y locales
- Declaración de arrays unidimensionales
- Declaración y definición de funciones
- Parámetros y llamadas a funciones
- Asignaciones
- Expresiones aritméticas
- Expresiones relacionales
- Expresiones lógicas
- Sentencias `if / else`
- Bucles `while`
- Bucles `for`
- Sentencias `break`
- Sentencias `continue`
- Sentencias `return`
- Salida por consola mediante `cout`

El compilador debía ser capaz de detectar errores léxicos, sintácticos y semánticos, diferenciar errores críticos de warnings, y producir salidas comprensibles para el usuario.

3. Diseño de la Solución

3.1 Arquitectura general del compilador

La arquitectura del compilador sigue una tubería secuencial compuesta por varias fases. Cada fase toma el resultado de la anterior y lo transforma o valida.

Flujo general:

Archivo fuente C++ → Lexer → Parser → Árbol sintáctico → Analizador semántico → Tabla de símbolos → Generación de código intermedio → Optimización → Archivos de salida

La clase principal `App.java` actúa como orquestador del proceso, controlando el flujo completo del compilador y coordinando la ejecución de cada etapa.

3.2 Descripción de cada fase de compilación

Análisis léxico

Se realiza mediante el lexer generado por ANTLR4 a partir de la gramática `MiLenguaje.g4`. Esta etapa transforma la secuencia de caracteres del archivo de entrada en tokens válidos del lenguaje. También se detectan caracteres no reconocidos y se reportan como errores léxicos.

Análisis sintáctico

Se realiza mediante el parser generado por ANTLR4. A partir de la secuencia de tokens, se verifica si el programa respeta la gramática definida. Si la sintaxis es correcta, se construye un árbol sintáctico que representa la estructura jerárquica del programa.

Visualización del árbol sintáctico

El compilador incorpora una opción gráfica (`--gui`) que permite mostrar el árbol sintáctico generado, utilizando la herramienta `TreeViewer` de ANTLR4.

Análisis semántico

Se implementó un visitante semántico específico que recorre el árbol sintáctico y valida aspectos como declaraciones, ámbitos, tipos, parámetros, retornos y uso correcto de arrays y funciones. Durante esta fase también se construye la tabla de símbolos.

Generación de código intermedio

Una vez validado el programa, se recorre nuevamente el árbol sintáctico para generar código de tres direcciones. Este código representa una forma intermedia más simple y cercana a una futura etapa de compilación más baja.

Optimización

Sobre el código intermedio se aplican técnicas de optimización que mejoran la salida eliminando instrucciones redundantes o simplificando expresiones cuando es posible.

3.3 Decisiones de diseño tomadas

Durante el desarrollo se adoptaron las siguientes decisiones:

- Utilizar ANTLR4 para generar automáticamente el lexer, el parser y la estructura base del visitor.
- Mantener separadas las responsabilidades principales del compilador:
 - `App.java` como orquestador
 - `AnalizadorSemantico.java` para reglas semánticas
 - `GeneradorCodigo.java` para código intermedio
 - `Optimizador.java` para coordinar optimizaciones
- Organizar las optimizaciones en clases separadas dentro de un paquete específico, para facilitar su mantenimiento y activación/desactivación.
- Implementar salida por colores ANSI para distinguir claramente éxitos, warnings y errores.
- Generar archivos auxiliares de salida junto al archivo fuente analizado, simplificando las pruebas.

4. Implementación

4.1 Detalles técnicos de la implementación

El proyecto fue desarrollado en Java, utilizando Maven como sistema de construcción y ANTLR4 como generador de analizadores.

La estructura principal del proyecto es la siguiente:

- `MiLenguaje.g4`: gramática del lenguaje
- `App.java`: punto de entrada y orquestación del flujo
- `AnalizadorSemantico.java`: validaciones semánticas
- `TablaSimbolos.java`: manejo de la tabla de símbolos
- `Simbolo.java`: representación de un símbolo
- `ResultadoSemantico.java`: contenedor de errores, warnings y tabla
- `GeneradorCodigo.java`: generación de código intermedio
- `Optimizador.java`: ejecución del pipeline de optimizaciones
- `optimizaciones/`: clases concretas de optimización

4.2 Descripción de la gramática ANTLR4

La gramática fue implementada en el archivo `MiLenguaje.g4`, que contiene tanto reglas léxicas como sintácticas.

Reglas léxicas

Las reglas léxicas definen los tokens del lenguaje, tales como:

- palabras reservadas (`if`, `else`, `while`, `for`, `return`, `break`, `continue`, `cout`)
- tipos (`int`, `float`, `double`, `char`, `string`, `bool`, `void`)
- operadores (`+`, `-`, `*`, `/`, `%`, `==`, `!=`, `>`, `<`, `>=`, `<=`, `&&`, `||`, `!`)
- delimitadores (`(`, `)`, `{`, `}`, `[`, `]`, `;`, `,`)
- identificadores
- números enteros y decimales
- literales de carácter
- cadenas de texto

Además, se incluyen reglas para comentarios y espacios en blanco, los cuales son descartados durante el análisis.

Reglas sintácticas

Las reglas sintácticas definen la estructura del lenguaje, incluyendo:

- programas
- funciones
- parámetros
- declaraciones
- asignaciones
- sentencias de control
- llamadas a funciones
- expresiones

Se definió una regla inicial **programa**, desde la cual parte el parser para analizar el archivo completo.

4.3 Detalles de la tabla de símbolos

La tabla de símbolos permite registrar y consultar la información semántica relevante de identificadores presentes en el programa.

Cada símbolo almacena:

- nombre
- tipo
- categoría (**variable**, **función**, **parámetro**)
- línea y columna de declaración
- ámbito
- tamaño de array, si corresponde
- lista de parámetros, en el caso de funciones

La tabla de símbolos se utiliza para:

- detectar declaraciones duplicadas
- controlar variables no declaradas
- validar parámetros de funciones
- diferenciar ámbitos globales y locales
- determinar tipos durante expresiones y asignaciones

4.4 Algoritmos utilizados en cada fase

Análisis léxico

El lexer generado por ANTLR recorre el flujo de caracteres y clasifica secuencias válidas según las reglas léxicas. Los caracteres no reconocidos son capturados y reportados.

Análisis sintáctico

El parser consume la secuencia de tokens y construye el árbol sintáctico según la gramática. Ante cualquier violación de estructura, genera errores sintácticos.

Análisis semántico

El análisis semántico se implementa mediante el patrón Visitor. El archivo `AnalizadorSemantico.java` recorre el árbol y aplica reglas como:

- declaración previa de variables y funciones
- compatibilidad de tipos en asignaciones
- compatibilidad de tipos en operaciones
- validación del índice de arrays
- control del tipo de retorno
- validación de cantidad y tipo de argumentos en llamadas
- warnings por variables locales no utilizadas

Generación de código intermedio

Se construye una lista de instrucciones de tres direcciones, usando:

- temporales (`t1`, `t2`, etc.)
- etiquetas (`THEN`, `END_IF`, `WHILE`, `FOR`, etc.)
- instrucciones de declaración
- asignaciones
- saltos condicionales e incondicionales
- retorno de funciones
- llamadas a funciones con parámetros

4.5 Técnicas de optimización implementadas

Se implementaron al menos tres técnicas de optimización, organizadas en clases separadas:

1. Simplificación de expresiones

Incluye:

- plegado de constantes
- eliminación de asignaciones redundantes

Ejemplo:

- $t1 = 5 + 3 \rightarrow t1 = 8$
- $x = x \rightarrow$ instrucción eliminada

2. Propagación de constantes

Cuando una variable recibe un valor constante, ese valor puede reemplazarse en expresiones posteriores mientras no haya cambios de flujo que invaliden la optimización.

Ejemplo:

- $a = 5$
- $b = a$
- resultado optimizado: $b = 5$

3. Eliminación de código muerto

Se eliminan instrucciones inalcanzables luego de instrucciones como `goto` o `return`, hasta encontrar una nueva etiqueta de control.

Estas técnicas mejoran la calidad del código intermedio generado y hacen más clara la salida final del compilador.

5. Ejemplos y Pruebas

5.1 Casos de prueba significativos

Durante el desarrollo se utilizaron distintos archivos de prueba para validar las distintas etapas del compilador:

Caso correcto

Archivo: `ejemplo_correcto.cpp`

Se utilizó un programa válido con:

- variables globales
- funciones
- arrays
- operaciones aritméticas
- condicionales
- retorno de valores

Este caso permitió comprobar el flujo completo:

- análisis léxico correcto
- análisis sintáctico correcto
- análisis semántico correcto
- generación de código intermedio
- optimización
- generación de archivos de salida

Caso con errores sintácticos

Archivo: `ejemplo_error.txt`

Este caso se utilizó para comprobar la detección de errores de estructura, tales como:

- sentencias mal cerradas
- uso incorrecto de símbolos
- faltas de punto y coma o paréntesis

Caso con errores semánticos

Archivo: `ejemplo_semantico_error.cpp`

Este archivo permitió verificar:

- variables duplicadas
- variables no declaradas
- warnings por variables no utilizadas

Casos extendidos

También se probaron ejemplos con:

- funciones con parámetros
- bucles `for` y `while`
- `break` y `continue`
- expresiones lógicas y relacionales
- llamadas a funciones
- asignaciones sobre arrays

5.2 Salidas generadas

El compilador genera dos archivos principales:

- `<nombre>_codigo_intermedio.txt`
- `<nombre>_codigo_optimizado.txt`

Además, muestra por consola:

- resumen del análisis léxico
- resumen del análisis sintáctico
- tabla de símbolos
- warnings semánticos
- errores semánticos, si existen
- código intermedio
- código optimizado
- resumen final de compilación

[Insertar aquí capturas de consola de un caso correcto]

[Insertar aquí capturas de consola de un caso con error semántico]

[Insertar aquí captura del árbol sintáctico con `--gui`]

5.3 Análisis de resultados

Los resultados obtenidos fueron satisfactorios en términos generales, ya que el compilador logra recorrer todas las fases requeridas por la consigna y produce diagnósticos claros sobre el estado del programa analizado.

Se comprobó que:

- los errores léxicos son detectados correctamente mediante la regla de captura de caracteres no reconocidos
- la gramática reconoce estructuras representativas del subconjunto definido
- el análisis semántico logra validar correctamente declaraciones, tipos y llamadas
- el código intermedio representa adecuadamente expresiones y estructuras de control
- las optimizaciones implementadas producen mejoras concretas en los casos apropiados

Asimismo, durante las pruebas se identificaron casos en los que fue necesario ajustar reglas de precedencia y validaciones para mejorar la consistencia del comportamiento frente a expresiones más complejas.

6. Dificultades Encontradas y Soluciones Aplicadas

Durante el desarrollo del proyecto surgieron diversas dificultades técnicas.

6.1 Definición de la gramática

Uno de los principales desafíos fue ampliar la gramática inicial para soportar funciones, arrays, bucles, sentencias de control y expresiones con distintos niveles de precedencia. Para resolverlo, se reorganizaron las reglas del parser y se probaron múltiples ejemplos hasta lograr una estructura estable.

6.2 Manejo de ámbitos

El control de ámbitos fue otro punto importante, ya que fue necesario diferenciar entre variables globales, locales y parámetros. Esto se resolvió implementando una tabla de símbolos con soporte de ámbito y búsquedas jerárquicas.

6.3 Validación semántica

La validación de tipos, retornos y llamadas a funciones implicó diseñar reglas semánticas manuales, ya que ANTLR resuelve léxico y sintaxis, pero no la semántica del lenguaje. Para ello se implementó un visitor semántico específico.

6.4 Generación de código intermedio

La traducción de estructuras como `if`, `while` y `for` a código de tres direcciones requirió el uso de temporales y etiquetas. La solución consistió en implementar una estrategia sistemática de generación de nombres temporales y saltos.

6.5 Organización del proyecto

A medida que el compilador fue creciendo, se volvió necesario modularizar responsabilidades. Por ese motivo, las optimizaciones se separaron en clases independientes dentro de un paquete específico, mejorando la mantenibilidad del código.

7. Conclusiones

El proyecto permitió aplicar de forma práctica los conceptos fundamentales de la materia, integrando en una misma herramienta las principales fases del proceso de compilación.

Se logró implementar un compilador funcional para un subconjunto de C++, con capacidad para:

- reconocer tokens válidos del lenguaje
- verificar estructura sintáctica
- analizar coherencia semántica
- construir y mostrar una tabla de símbolos
- generar código intermedio
- aplicar optimizaciones básicas
- mostrar resultados comprensibles para el usuario

Además del valor técnico, el trabajo permitió comprender la importancia de la separación de responsabilidades entre fases, el uso de gramáticas formales y la relación entre teoría de compiladores y construcción de herramientas reales.

Como posibles extensiones futuras, el proyecto podría ampliarse para incorporar:

- nuevas estructuras del lenguaje
- más técnicas de optimización
- generación de código destino más cercana a ensamblador
- construcción explícita de un AST propio
- soporte para arreglos multidimensionales o estructuras más avanzadas

8. Referencias Bibliográficas

- Github de ANTLR4. <https://github.com/antlr/antlr4>
- Material teórico y práctico de la cátedra Técnicas de Compilación.
- ChatGPT, Claude